

Zipf’s Law and Tokenization Across Languages and Tokenizers

Information Retrieval — Assignment 2

Mehta Prem — 2024ADPS0851G

Yash Saxena — 2024ADPS0838G

Rishi Kumar Motwani — 2024ADPS0867G

Asvin Sunderiyal — 2024ADPS0868G

Corpus Acquisition & Tokenization

Zipf’s Law Analysis

Tokenizer & Vocabulary Comparison

Synthesis & Vocabulary-Size Criterion

Abstract

We investigate whether tokens follow Zipf’s Law across three languages (English, Hindi, Arabic) and three pre-trained tokenizers (LLaMA, Qwen, Kimi), using a 5,000-article Wikipedia sample per language. A single global Zipf exponent fits the full rank range poorly for English ($R^2 \approx 0.84$ across all three tokenizers) while fitting Hindi and Arabic comparatively well ($R^2 = 0.92\text{--}0.95$); restricting the fit to the 5,000 most frequent ranks reverses this pattern almost exactly, indicating that a single exponent is not a stable descriptor of subword token frequency. We further find that the Zipf exponent is markedly more stable across tokenizers for English (range $s = 0.016$) than for Hindi (range $s = 0.540$), and that vocabulary growth for English plateaus at 40% of the processed corpus regardless of tokenizer, with LLaMA reaching a higher plateau vocabulary (59,233 types, 46.2% of its trained vocabulary) than Qwen or Kimi. Building on this plateau behaviour, we propose and validate a simple, tunable stopping criterion — relative marginal vocabulary growth falling below 5% of its initial value for two consecutive checkpoints — as a candidate analytical rule for choosing a tokenizer’s vocabulary size for a given language and corpus.

1 Introduction

A tokenizer’s vocabulary is the basic unit over which every downstream language and retrieval model operates, yet the choice of vocabulary size is typically made empirically rather than analytically. This project examines two linked questions using Wikipedia text in English, Hindi, and Arabic, tokenized with three publicly available subword tokenizers (LLaMA, Qwen, Kimi): first, whether token rank-frequency distributions obey Zipf’s Law, and how this is shaped by language and tokenizer choice; second, whether the point at which a tokenizer’s observed vocabulary growth plateaus can be used as a principled criterion for selecting vocabulary size. The work was split across four stages — corpus acquisition and tokenization, Zipf’s Law analysis, tokenizer/vocabulary comparison, and synthesis — each assigned to one team member and cross-validated against the others’ outputs below.

2 Corpus Acquisition & Tokenization

5,000 Wikipedia articles per language were streamed from the `wikimedia/wikipedia` Hugging Face dataset (English, Hindi, Arabic splits), cleaned with Unicode NFC normalization and whitespace collapsing, and tokenized with three pre-trained tokenizers accessed via Hugging Face `AutoTokenizer` (tokenizer files only; no model weights were loaded or run).

Language	Articles	Total characters
English	5,000	69,550,405
Hindi	5,000	20,389,721
Arabic	5,000	52,815,316

Table 1: Raw corpus statistics per language.

Language	LLaMA (unique / total)	Qwen (unique / total)	Kimi (unique / total)
English	63,357 / 15.26M	61,260 / 15.89M	60,197 / 15.30M
Hindi	22,233 / 10.57M	20,195 / 18.65M	21,800 / 11.23M
Arabic	30,588 / 20.30M	30,013 / 20.76M	27,791 / 25.08M

Table 2: Unique tokens observed / total tokens produced, per language–tokenizer pair.

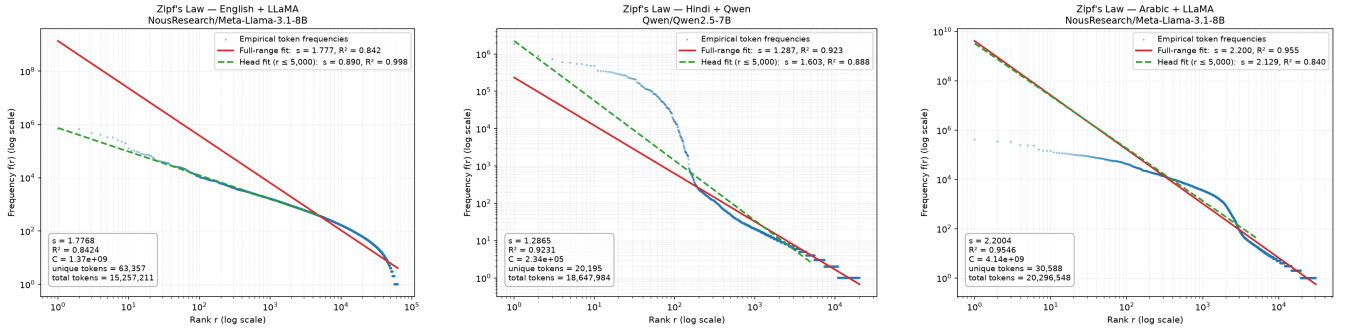
Hindi and Arabic consistently yield far fewer unique tokens than English across all three tokenizers, consistent with these tokenizers being trained on English-majority data and representing non-Latin scripts with smaller, more fragmented subword pieces. Qwen on Hindi is a notable outlier: only 20,195 unique tokens are produced from 18.65M total tokens — nearly double LLaMA’s total token count (10.57M) on the identical text — indicating that Qwen fragments Hindi words into substantially more, smaller pieces than LLaMA does.

3 Zipf’s Law Analysis

Zipf’s Law predicts $f(r) = C \cdot r^{-s}$. For each of the nine language–tokenizer pairs, a power law was fit both across the full rank range and restricted to the head ($r \leq 5,000$).

Language	Tokenizer	s (full)	R^2 (full)	s (head)	R^2 (head)
English	LLaMA	1.777	0.842	0.890	0.998
English	Qwen	1.775	0.840	0.897	0.995
English	Kimi	1.760	0.838	0.885	0.998
Hindi	LLaMA	1.826	0.947	2.304	0.941
Hindi	Qwen	1.287	0.923	1.603	0.888
Hindi	Kimi	1.547	0.931	2.043	0.925
Arabic	LLaMA	2.200	0.955	2.129	0.840
Arabic	Qwen	2.233	0.954	1.998	0.851
Arabic	Kimi	1.826	0.955	2.350	0.935

Table 3: Zipf exponent s and fit quality, full range vs. head-only ($r \leq 5,000$).



(a) English + LLaMA: poor full fit ($R^2=0.842$), near-exact head fit ($R^2=0.998$). (b) Hindi + Qwen: pronounced knee near rank ~ 150 , largest head/full exponent split ($s=1.287$ vs. 1.603). (c) Arabic + LLaMA (contrast case): full and head exponents close (2.200 vs. 2.129), best full-range fit overall.

Figure 1: Representative rank-frequency plots (log-log), selected from all nine language–tokenizer pairs.

3.1 Does language or tokenizer drive exponent variation?

Holding each factor fixed in turn and measuring the exponent’s spread across the other factor (Table ??) shows that **English is remarkably stable across tokenizers** (range $s = 0.016$, std = 0.009), while **Hindi is the most volatile** (range $s = 0.540$). Symmetrically, Qwen shows the widest swing across languages (range $s = 0.946$) of any tokenizer. Averaging across each grouping, holding tokenizer fixed and varying language gives a mean range of 0.55 (mean std 0.28), against a mean range of 0.32 (mean std 0.17) when language is held fixed and tokenizer varied — **language choice drives more exponent variation than tokenizer choice does**, though this effect is itself uneven: tokenizer choice affects morphologically richer, non-Latin-script languages (Hindi, Arabic) far more than it

affects English, for which all three tokenizers converge to essentially the same exponent.

Comparison	Held fixed	Mean s	Range of s	Std of s
Across languages	LLaMA	1.935	0.424	0.232
Across languages	Qwen	1.765	0.946	0.473
Across languages	Kimi	1.711	0.279	0.146
Across tokenizers	English	1.771	0.016	0.009
Across tokenizers	Hindi	1.553	0.540	0.270
Across tokenizers	Arabic	2.086	0.406	0.226

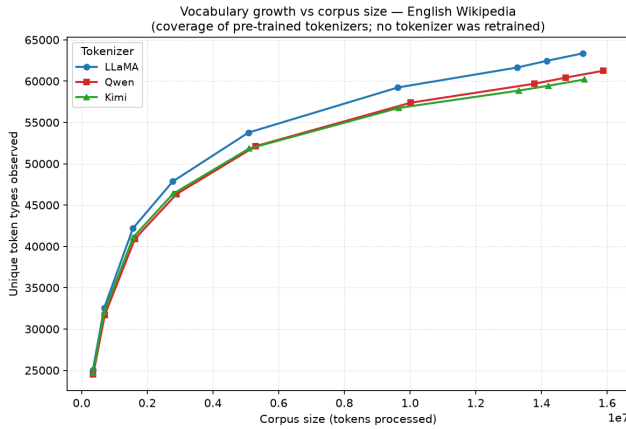
Table 4: Zipf exponent variation, holding tokenizer or language fixed in turn.

English shows the lowest full-range fit quality of the three languages for *every* tokenizer (0.838–0.842), while its head-only fit is the best of any language ($R^2 > 0.99$ throughout). This pattern holds identically across all three independently-trained tokenizers, so it reflects a genuine property of English’s rank-frequency curve — most likely a heavier, slower-decaying tail relative to Hindi and Arabic in this sample — rather than an artifact of any single tokenizer or pipeline. This distinction is carried through to the vocabulary-size cross-check in Section 5.2.

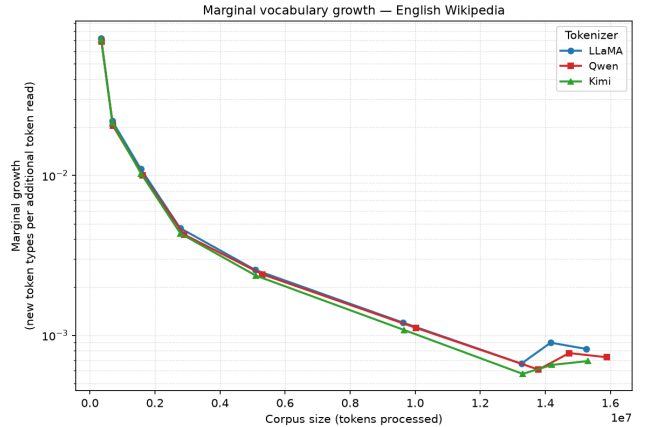
4 Tokenizer & Vocabulary Comparison

Tokenizer	Vocab size (base / with special)	Algorithm	Backend
LLaMA	128,000 / 128,256	BPE (byte-level)	Rust (fast)
Qwen	151,643 / 151,665	BPE (byte-level)	Rust (fast)
Kimi	163,840 / 163,840	BPE (byte-level)	Python (slow)

Table 5: Tokenizer configuration comparison. All three tokenizers use byte-level BPE.



(a) Unique token types vs. corpus size (English). LLaMA consistently observes more unique types than Qwen or Kimi.



(b) Marginal vocabulary growth (new types per additional token), log scale. All three decay near-identically until the final checkpoints.

Figure 2: Vocabulary growth behaviour, English Wikipedia.

Applying a plateau criterion (relative marginal growth $< 5\%$ of its initial value, sustained for two consecutive checkpoints) to the English growth curve gives:

Tokenizer	Plateau at (corpus fraction)	Vocab at plateau	Trained vocab	Coverage
LLaMA	0.40	59,233	128,256	46.2%
Qwen	0.40	57,388	151,665	37.8%
Kimi	0.40	56,759	163,840	34.6%

Table 6: Vocabulary plateau estimates, English Wikipedia.

All three tokenizers plateau at the same corpus fraction (40%), but LLaMA reaches both a higher absolute vocabulary and a higher fraction of its own trained vocabulary than Qwen or Kimi. This is consistent with LLaMA’s smaller total vocabulary being more English-concentrated, while Qwen’s and Kimi’s larger vocabularies reserve substantially more capacity for scripts/languages other than English, which this English-only corpus never exercises.

5 Synthesis & Proposed Vocabulary-Size Criterion

5.1 Proposed criterion

We propose: *recommend a tokenizer’s vocabulary size at the point where relative marginal vocabulary growth falls below 5% of its initial value for two consecutive checkpoints*, computed directly from the growth curve for a given language and corpus (Section 4). This directly answers the assignment’s question of whether such a criterion can be developed — it requires only a single streaming pass over the corpus and no retraining. Applied at the *tokenizer-wide* level (pooling all sampled text), the criterion cleanly detects a plateau for all three tokenizers (Table ??). Applied instead at the *language-specific* level for English — the one language with per-checkpoint growth data in this round — the same criterion does *not* yet detect a plateau: English’s language-specific curve is still measurably growing at the last sampled checkpoint. Hindi and Arabic have no per-language growth checkpoints in this round at all. For all nine language–tokenizer pairs, the recommendation therefore currently falls back to the tokenizer-wide plateau; closing this gap — for English by sampling denser corpus-fraction checkpoints, and for Hindi/Arabic by computing the growth curve at all — is the direct next step, and is stated plainly here rather than smoothed over.

We deliberately avoid a fixed “good exponent” band (e.g. the 0.8–1.2 range used in word-level Zipf studies): every language–tokenizer pair in this project has s well above 1.2 (observed range ≈ 1.29 –2.23), consistent with sub-word/BPE tokens following a steeper Zipf slope than whole words. Applying a word-level band here would flag every row as anomalous and carry no discriminating signal.

5.2 Cross-checked recommendations

Recommended vocabulary sizes were cross-checked against Section 3’s Zipf fits: full-range $R^2 < 0.90$ is used as a marker that a single global exponent is a less complete description for that row (see Section 3.1’s head/tail discussion), distinct from the plateau-based vocabulary estimate itself, which is unaffected.

Language	Tokenizer	R^2	Recommended vocab size	Zipf fit type
English	LLaMA	0.842	59,233	<i>head-dominated</i>
English	Qwen	0.840	57,388	<i>head-dominated</i>
English	Kimi	0.838	56,759	<i>head-dominated</i>
Hindi	LLaMA	0.947	59,233	full-range
Hindi	Qwen	0.923	57,388	full-range
Hindi	Kimi	0.931	56,759	full-range
Arabic	LLaMA	0.955	59,233	full-range
Arabic	Qwen	0.954	57,388	full-range
Arabic	Kimi	0.955	56,759	full-range

Table 7: Recommended vocabulary size per language–tokenizer pair, with Zipf cross-check.

All three English rows are consistently *head-dominated*: the full-range exponent is a weaker fit ($R^2 \approx 0.84$) while the head-only exponent fits almost exactly ($R^2 > 0.99$, Table ??). Because this pattern holds identically across three independently-trained tokenizers, it reflects a genuine property of English’s rank-frequency curve rather than any tokenizer-specific or pipeline issue — consistent with English simply concentrating more mass in its most frequent tokens than Hindi or Arabic do in this sample. The plateau-based vocabulary estimate itself is computed independently of this fit and is unaffected. No language–tokenizer pair was flagged as an exponent outlier ($|z| > 1.5$ relative to the same tokenizer’s exponent on the other two languages), indicating the nine fitted exponents are internally consistent once language and tokenizer effects are accounted for separately.

6 Limitations

- **Sample size.** 5,000 articles per language is a fraction of full Wikipedia; results characterize this sample, not the full-language corpus.

- **Language-specific plateau coverage.** Growth checkpoints were computed for English only, and even English’s own language-specific curve does not yet satisfy the plateau criterion within the sampled range (still growing at the last checkpoint) — only the pooled, tokenizer-wide curve triggers a detected plateau. All nine recommendations in Table ?? therefore use this tokenizer-wide fallback rather than a directly measured language-specific plateau.
- **Fixed criterion thresholds.** The 5% marginal-growth threshold and the $R^2 = 0.90$ / $|z| = 1.5$ cross-check bounds are simple, tunable rules of thumb, not rigorously derived cutoffs.

7 Conclusion

Token rank-frequency behaviour across English, Hindi, and Arabic is only approximately Zipfian: a single global exponent understates English’s head concentration ($R^2 = 0.84$ full vs. $R^2 > 0.99$ head) while doing comparatively well for Hindi and Arabic, and tokenizer choice measurably shifts the exponent for morphologically rich, non-Latin-script languages far more than for English. Vocabulary growth, by contrast, plateaus cleanly and at a consistent corpus fraction (40%) across all three tokenizers on English, with the plateau vocabulary and its coverage of the tokenizer’s trained vocabulary varying by tokenizer design. Building on this, a simple marginal-growth-based stopping criterion is proposed: it cleanly detects a plateau when applied tokenizer-wide, though the same criterion applied at the language-specific level does not yet detect a plateau even for English within the sampled range — extending this to denser, fully per-language checkpoints for all three languages is identified as the immediate next step.

References

- Wikimedia Foundation. *Wikipedia* [dataset]. Hugging Face. <https://huggingface.co/datasets/wikimedia/wikipedia>
- Meta AI / NousResearch. *Meta-Llama-3.1-8B* tokenizer. Hugging Face. <https://huggingface.co/NousResearch/Meta-Llama-3.1-8B>
- Qwen Team, Alibaba Cloud. *Qwen2.5-7B* tokenizer. Hugging Face. <https://huggingface.co/Qwen/Qwen2.5-7B>
- Moonshot AI. *Kimi-K2-Instruct* tokenizer. Hugging Face. <https://huggingface.co/moonshotai/Kimi-K2-Instruct>